Universidad de San Buenaventura – Cali

Laboratorio de Software III | 2026

Primera Entrega Avances

Plataforma de Práctica de Programación

David Steven Hernandez Garces

Tabla de Contenido

Tabla de Contenido 2
1. Github 3
2. Estructura 3
3. ADRS Del Proyecto 4
4. Definition of Done 8
5. ReadMe 10
6. Diagramas 13
7. Tablero Kanban 16

1. Github

https://github.com/davids03/leetcode-usb/tree/main

2. Estructura

```
leetcode-usb/
├── backend/
│ ├── src/
│ │ ├── app.py
│ │ ├── extensions.py
│ │ ├── models/
│ │ ├── routes/
│ │ └── services/
│ ├── .env (no subido)
│ ├── requirements.txt
│ └── venv/ (ignorado)
├── frontend/
│ ├── src/
│ │ ├── pages/
│ │ ├── services/
│ │ └── App.jsx
│ ├── package.json
│ └── ...
├── docs/
│ ├── adr/ (Decisiones arquitectónicas)
│ ├── diagrams/ (Diagramas C4 en PlantUML)
│ └── definition-of-done.md
├── scripts/
│ └── seed_db.py
├── .gitignore
└── README.md
```

3. ADRS Del Proyecto

 # ADR-001: Uso de Flask como framework backend

Estado
Aceptado

Contexto
La plataforma requiere una API REST que maneje autenticación, gestión de problemas, ejecución de código y casos de prueba. El docente estableció **Python** como lenguaje de backend, pero no especificó un framework obligatorio. El equipo debe elegir entre Flask, Django REST Framework (DRF) y FastAPI.

Criterios considerados:
- Curva de aprendizaje para estudiantes de primer año.
- Tamaño del proyecto (alcance del corte dos: 4 HUs).
- Facilidad de integración con MySQL, JWT y Judge0.
- Legibilidad del código para revisiones académicas.

Decisión
Se utilizará **Flask** con las siguientes extensiones:
- `Flask-SQLAlchemy` para ORM.
- `Flask-JWT-Extended` para autenticación stateless.
- `Flask-CORS` para comunicación con frontend React.
- `Flask-Bcrypt` para hashing de contraseñas.
- `python-dotenv` para configuración.

La API se organizará en blueprints: `auth`, `problems`, `submissions`, `test_cases`.

Consecuencias
Positivas:
- Código explícito y minimalista, ideal para entender cada capa (rutas, controladores, servicios).
- Integración directa con SQLAlchemy y Judge0 sin capas adicionales.
- El equipo puede desarrollar rápidamente porque Flask no impone estructura rígida.
- Cumple exactamente el lineamiento del docente.

Negativas:
- Mayor responsabilidad en estructura y seguridad (no hay protección contra CSRF por defecto, se debe configurar CORS adecuadamente).
- Para alta concurrencia (futuro escalamiento), Flask síncrono podría requerir workers adicionales (Gunicorn con gevent o multiproceso).

Opciones consideradas
- **Django REST Framework**: ORM potente, admin integrado, autenticación completa. Descartado por su curva de aprendizaje y sobreingeniería para el alcance actual.
- **FastAPI**: Asíncrono, documentación automática, validación con Pydantic. Descartado por no ser el framework solicitado por el docente y por menor alineación con el perfil académico.

ADR-002: Uso de MySQL como motor de base de datos

Estado
Aceptado

Contexto
La plataforma necesita persistir usuarios, problemas, casos de prueba, envíos y resultados. Se requiere:
- Integridad referencial (claves foráneas).
- Consultas eficientes (índices por usuario, problema, estado).
- Soporte para datos semiestructurados (resultados de cada caso de prueba).
- Migraciones versionadas (aunque en el corte dos se usa `db.create_all()` por simplicidad).

Las alternativas evaluadas son: MySQL, PostgreSQL, SQLite (solo desarrollo) y MongoDB (NoSQL).

Decisión
Se elige **MySQL (versión 8.0)** como motor de base de datos, **SQLAlchemy** como ORM y (opcionalmente) **Alembic** para migraciones. El modelo de datos incluye `User`, `Problem`, `TestCase`, `Submission`, `SubmissionResult` y `RefreshToken`. Se usa el tipo `JSON` de MySQL para almacenar resultados detallados por caso (aunque en la práctica se usan columnas separadas por simplicidad).

La cadena de conexión se define en variable de entorno `DATABASE_URL` con el dialecto `mysql+pymysql://`. En desarrollo local se puede usar SQLite cambiando la URL, gracias a la abstracción de SQLAlchemy.

Consecuencias
Positivas:
- MySQL es ampliamente adoptado, con gran cantidad de documentación y soporte comunitario.
- Excelente rendimiento en operaciones de lectura intensiva, típicas de una plataforma donde los estudiantes consultan problemas y ven resultados.
- El tipo `JSON` permite almacenar resultados de ejecución sin esquema fijo, manteniendo flexibilidad.
- La combinación MySQL + InnoDB garantiza integridad referencial y transacciones ACID.

Negativas:
- MySQL tiene algunas diferencias con el estándar SQL (por ejemplo, manejo de `ON DELETE CASCADE` requiere InnoDB, que ya está asegurado).
- En configuraciones por defecto, puede tener problemas con `utf8mb4` para emojis o caracteres especiales; se debe configurar explícitamente la codificación.
- La concurrencia de escritura (envíos de código) es moderada; MySQL maneja bien hasta miles de transacciones por segundo.

Opciones consideradas
- **PostgreSQL**: Mayor cumplimiento de estándar SQL, JSONB más potente. Descartado por menor familiaridad del equipo y porque MySQL es suficiente para el alcance.
- **SQLite**: Sin servidor, archivo único, ideal para desarrollo. Descartado para producción por falta de concurrencia real.

- **MongoDB**: Sin integridad referencial nativa, consultas complejas entre colecciones. Descartado porque el modelo es mayormente relacional.

ADR-003: Autenticación stateless con JWT

Estado
Aceptado

Contexto
La HU1 exige inicio de sesión seguro, sesión con tiempo de expiración definido y mensajes de error genéricos. Se consideraron:
- Sesiones tradicionales con cookies y almacenamiento en servidor (Redis o base de datos).
- OAuth2 externo (Google, GitHub) – fuera de alcance para primer corte.
- JWT (JSON Web Tokens) stateless.

Decisión
Se implementa **JWT** mediante la extensión `Flask-JWT-Extended`. El flujo:
1. `POST /api/auth/login` recibe username y password.
2. Verifica hash bcrypt almacenado.
3. Genera un `access_token` con identidad del usuario y expiración configurable (1 hora).
4. El cliente almacena el token en `localStorage` (con medidas de seguridad).
5. Las peticiones protegidas incluyen el token en cabecera `Authorization: Bearer <token>`.
6. El decorador `@jwt_required()` valida el token antes de ejecutar el endpoint.
7. Los mensajes de error se capturan y retornan como "Credenciales inválidas" sin detalles técnicos.

Se incluye soporte para `refresh_token` (opcional) almacenado en base de datos con hash para permitir revocación.

Consecuencias
Positivas:
- Stateless: el servidor no necesita almacenar sesiones, facilitando escalabilidad horizontal.
- El token firmado garantiza integridad (no puede ser modificado sin invalidarse).
- Expiración automática cumple el criterio de aceptación.
- Fácil de implementar con Flask-JWT-Extended.

Negativas:
- El token no puede ser invalidado antes de su expiración a menos que se implemente una lista negra. Se mitiga con refresh tokens rotativos y vida corta del access token.
- Almacenar token en `localStorage` expone a XSS; se recomienda usar `httpOnly cookies` en producción, pero requiere manejo CSRF.

Opciones consideradas
- **Sesiones con Redis**: Revocación inmediata, control total, pero requiere estado en servidor y más complejidad. Descartado por simplicidad inicial.
- **OAuth2**: Estándar industrial, pero requiere proveedor externo y no es necesario para usuarios internos de la USB.

ADR-004: Estrategia de ejecución de código (simulación inicial, migración futura a Judge0)

Estado
Aceptado (con planes de evolución)

Contexto
La HU3 y HU4 requieren ejecutar código enviado por estudiantes de forma segura y aislada, mostrando errores de compilación, ejecución y resultados contra casos de prueba. Durante el desarrollo, la configuración de Judge0 en Windows presentó problemas persistentes (sandbox `/box` no accesible). Se necesita una solución que permita avanzar con la lógica del frontend y backend, sin bloquear el proyecto.

Decisión
Fase 1 (desarrollo inmediato): Implementar un **servicio simulado (mock)** en `judge0_service.py` que devuelva resultados predecibles sin ejecutar código real. Este mock permite probar el flujo completo (autenticación, envío, guardado en base de datos, visualización de resultados) sin depender de infraestructura externa. La simulación incluye:
- Éxito (`accepted`) para la mayoría de los casos.
- Errores simulados (`compile_error`, `runtime_error`) si el código está vacío o contiene palabras clave como "error".

Fase 2 (futuro): Migrar a una solución real de ejecución de código. Las alternativas consideradas son:
- **Judge0 (oficial)**: Requiere WSL2 o Linux para funcionar correctamente en Windows. Se intentó pero no se logró estabilizar en el tiempo disponible.
- **Piston API**: Más ligera y fácil de auto-alojar, con soporte para múltiples lenguajes.
- **Servicio en la nube (Judge0 CE demo)**: Usar la API pública `https://ce.judge0.com` para pruebas, con límites de uso.

Consecuencias
Positivas (simulación):
- Permite completar el desarrollo del frontend y backend sin bloqueos.
- Fácil de modificar para probar diferentes escenarios (errores, tiempos de espera).
- No requiere recursos adicionales (Docker, WSL2).

Negativas (simulación):
- No ejecuta código real, por lo que no se puede validar la corrección de las soluciones de los estudiantes en un entorno real.
- No es adecuada para producción o demostración final con funcionalidad completa.

Plan de migración:
- Una vez que el proyecto esté funcional, se dedicará tiempo a configurar correctamente Judge0 en WSL2 (o usar Piston API).
- Se reemplazará el mock por el servicio real sin modificar la interfaz del endpoint, asegurando compatibilidad.

Opciones consideradas

- **Judge0 nativo en Windows**: Descartado por problemas de compatibilidad del sandbox (`isolate`).
- **Docker con WSL2**: No se logró estabilizar en el tiempo disponible, pero es la ruta futura recomendada.
- **Piston API**: Alternativa prometedora, se evaluará en la siguiente fase.

4. Definition of Done

 # Definition of Done (DoD) – Sprint Corte Dos

Una **historia de usuario** o **tarea técnica** se considera **TERMINADA** solo cuando cumple **todos** los siguientes criterios.

1. Criterios funcionales (basados en las HUs)

- [ ] **La implementación satisface todos los criterios de aceptación** de la HU correspondiente (según el documento de requisitos).
- [ ] **Las validaciones de entrada y salida** se manejan según lo especificado:
  - Mensajes de error genéricos en login (no se revelan detalles de seguridad).
  - Errores de compilación/ejecución sin exposición de rutas internas.
- [ ] **Los casos de prueba públicos** (HU4) se ejecutan correctamente contra la solución del estudiante, mostrando estado "Aprobado/Fallo" por cada caso.
- [ ] **El flujo completo** (inicio de sesión → listado de problemas → detalle con ejemplos → ejecución simulado) funciona sin intervención manual en el entorno de desarrollo.

2. Criterios técnicos y de calidad

- [ ] **El código pasa las pruebas automatizadas** definidas para esa HU (unitarias e integración). Al menos:
  - Pruebas de endpoints críticos (auth, problemas, envío simulado).
  - Pruebas de seguridad básica (inyección SQL, XSS en editor).
- [ ] **No hay errores de compilación o linting**:
  - Frontend: ESLint (sin errores).
  - Backend: Flake8 o equivalente (sin errores graves).
- [ ] **La base de datos** tiene el esquema actualizado (usando `db.create_all()` o migraciones Alembic) y refleja los modelos sin conflictos.
- [ ] **El código sigue los estándares de estilo** del proyecto:
  - Python: PEP8.
  - JavaScript/React: Airbnb o estándar recomendado.
- [ ] **No hay vulnerabilidades conocidas** en dependencias críticas (revisión con `npm audit` y `safety check`).

---

3. Criterios de documentación

- [ ] **La funcionalidad nueva está documentada** en el `README.md` (si es relevante para el usuario o desarrollador).
- [ ] **Los endpoints nuevos** están documentados (al menos en comentarios del código o en un archivo `API.md`).
- [ ] **Se actualizó el diagrama C4** si la arquitectura cambió (no aplica para este corte, pero se verifica).
- [ ] **Los ADRs relevantes** se crearon o actualizaron (ej: decisión de simular ejecución de código).

---

4. Criterios de integración y despliegue

- [ ] **La rama de la característica** se integró a `develop` (o `main`) mediante **pull request** revisado por al menos otro miembro del equipo.
- [ ] **La integración continua (CI)** pasa exitosamente (si está configurada: pruebas, linting).
- [ ] **La funcionalidad se probó en entorno local** con el stack completo (Flask, MySQL, React).
- [ ] **No hay regresiones** en funcionalidades ya terminadas (se ejecutan pruebas de humo manuales).

---

5. Criterios de experiencia de usuario (UX)

- [ ] **Los mensajes de error** son legibles, no técnicos y no revelan información interna.
- [ ] **La interfaz responde** en menos de 2 segundos para operaciones habituales (listado, detalle de problema, ejecución simulada).
- [ ] **El editor de código** (Monaco) muestra resaltado de sintaxis, numeración de líneas y permite escribir/editar cómodamente.
- [ ] **La navegación** entre páginas (login → problemas → detalle → volver) es intuitiva y mantiene el estado de autenticación.

---

6. Criterios de seguridad (específicos del proyecto)

- [ ] **Las contraseñas** se almacenan con hash bcrypt (costo 12).
- [ ] **Los JWT** tienen tiempo de expiración configurado (`JWT_ACCESS_TOKEN_EXPIRES`) y se validan en cada endpoint protegido.
- [ ] **La ejecución de código** (aunque simulada) no permite acceso al sistema de archivos ni comandos del servidor. En la simulación, no se ejecuta código real.
- [ ] **Las variables sensibles** (secret keys, contraseñas DB) se gestionan con `.env` y no están en el repositorio.

- [ ] **CORS** está configurado para permitir solo el origen del frontend en desarrollo (`http://localhost:5173`).

7. Aceptación final por el Product Owner

- [ ] **El product owner (docente o representante)** ha verificado la HU en una demo y la acepta como completada.
- [ ] **Los criterios de aceptación** definidos en las HUs se cumplen al 100% (o se justifica técnicamente si algún criterio se posterga).

Notas adicionales

- La DoD puede evolucionar durante el sprint si el equipo identifica nuevos criterios, pero cualquier cambio debe ser consensuado y documentado.
- Para tareas técnicas que no son HU (ej: configuración de Judge0, ajustes de CORS), se aplican los criterios 2, 3, 4 y 6, más la verificación de que la tarea cumple su objetivo definido en el tablero Kanban.
- En caso de usar simulación de ejecución de código (ADR-004), se acepta temporalmente que no se ejecute código real, siempre que la interfaz y el flujo de datos funcionen correctamente.

5. ReadMe

Plataforma de Práctica de Programación USB – LeetCode-like
Proyecto desarrollado para el curso **Laboratorio de Software III** (Universidad de San Buenaventura, Cali).
Inspirado en LeetCode, permite a los estudiantes autenticarse, ver problemas organizados por dificultad, escribir código en un editor Monaco y recibir retroalimentación (simulada por ahora).
Tabla de Contenidos

- Tecnologías
- Estructura del proyecto
- Requisitos previos
- Instalación y ejecución (backend)
- Instalación y ejecución (frontend)
- Endpoints principales
- Variables de entorno
- Nota sobre la ejecución de código
- Autores

Tecnologías

Capa	Tecnología
Backend	Python 3.11 + Flask
Base de datos	MySQL 8.0 + SQLAlchemy (PyMySQL)
Autenticación	JWT (Flask-JWT-Extended)
Frontend	React 18 + Vite + Monaco Editor
Estilos	CSS plano (puede mejorarse)
Entorno	Python venv, npm

Estructura del proyecto

```
leetcode-usb/
├── backend/
│   ├── src/
│   │   ├── app.py
│   │   ├── extensions.py
│   │   ├── models/
│   │   ├── routes/
│   │   └── services/
│   ├── .env (no subido)
│   ├── requirements.txt
│   └── venv/ (ignorado)
├── frontend/
│   ├── src/
│   │   ├── pages/
│   │   ├── services/
│   │   └── App.jsx
│   ├── package.json
│   └── ...
├── docs/
│   ├── adr/ (Decisiones arquitectónicas)
│   ├── diagrams/ (Diagramas C4 en PlantUML)
│   └── definition-of-done.md
├── scripts/
│   └── seed_db.py
├── .gitignore
└── README.md
```

Requisitos previos

- **Python 3.11** (recomendado) – Descargar
- **MySQL 8.0** – Descargar
- **Node.js 18+** – Descargar
- **Git** – Descargar

Instalación y ejecución (backend)

1. Clonar el repositorio

2. Crear entorno virtual y activarlo

cd backend python -m venv venv venv\Scripts\activate # Windows PowerShell

3. Instalar dependencias

pip install -r requirements.txt

4. Configurar variables de entorno Crea el archivo backend/.env con el siguiente contenido (ajusta las credenciales de MySQL):

FLASK_APP=src/app.py FLASK_ENV=development SECRET_KEY=clave_super_secreta_123 JWT_SECRET_KEY=otra_clave_para_jwt_456 DATABASE_URL=mysql+pymysql://leetcode_user:Segura123@localhost/leetcode_usb JUDGE0_URL=http://localhost:2358 # (no necesario para simulación)

5. Crear base de datos y usuario en MySQL

CREATE DATABASE leetcode_usb; CREATE USER 'leetcode_user'@'localhost' IDENTIFIED BY 'Segura123'; GRANT ALL PRIVILEGES ON leetcode_usb.* TO 'leetcode_user'@'localhost'; FLUSH PRIVILEGES;

6. Ejecutar el backend

python -m src.app

7. (Opcional) Poblar la base de datos con problemas de ejemplo Desde otra terminal (o deteniendo el servidor):

cd .. python scripts/seed_db.py

Instalación y ejecución (frontend)

1. Ir a la carpeta frontend e instalar dependencias

cd frontend npm install npm install axios @monaco-editor/react react-router-dom

2. Ejecutar el servidor de desarrollo npm run dev El frontend estará en http://localhost:5173.
3. Uso básico Regístrate o inicia sesión con un usuario creado (puedes hacerlo desde la interfaz o con Postman). Explora la lista de problemas, filtra por dificultad. Haz clic en un

problema para ver su descripción y casos de prueba públicos. Escribe código en el editor Monaco y presiona Ejecutar. Los resultados (simulados) se mostrarán en el panel derecho.

Endpoints principales

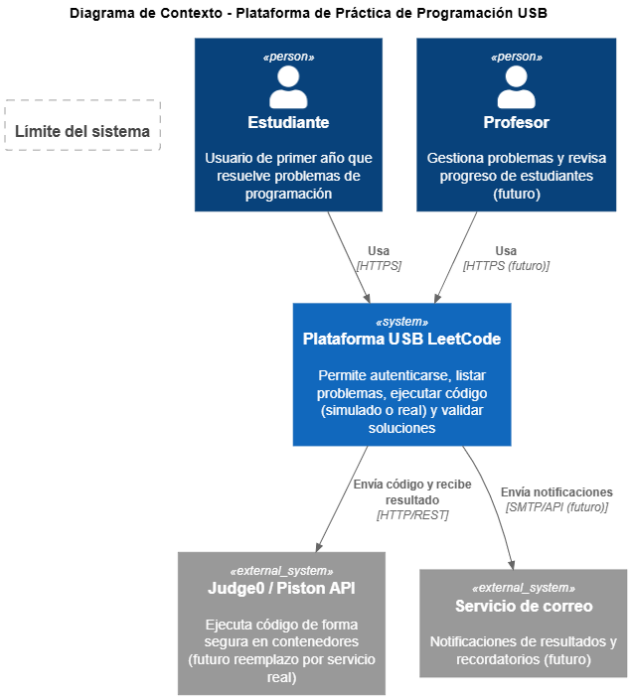
Método Endpoint Descripción Autenticación POST /api/auth/register Registrar nuevo usuario No POST /api/auth/login Iniciar sesión, devuelve JWT No GET /api/problems/ Listar problemas (filtro opcional) JWT GET /api/problems/:id Detalle de problema + casos públicos JWT POST /api/submissions/ Enviar código para evaluación (simulado) JWT
Variables de entorno (backend)

Variable Propósito Ejemplo DATABASE_URL Conexión a MySQL mysql+pymysql://user:pass@localhost/db SECRET_KEY Clave para sesiones de Flask clave_super_secreta_123 JWT_SECRET_KEY Clave para firmar tokens JWT otra_clave_para_jwt_456 JUDGE0_URL URL del servicio de ejecución (mock) http://localhost:2358 (no necesario para mock)

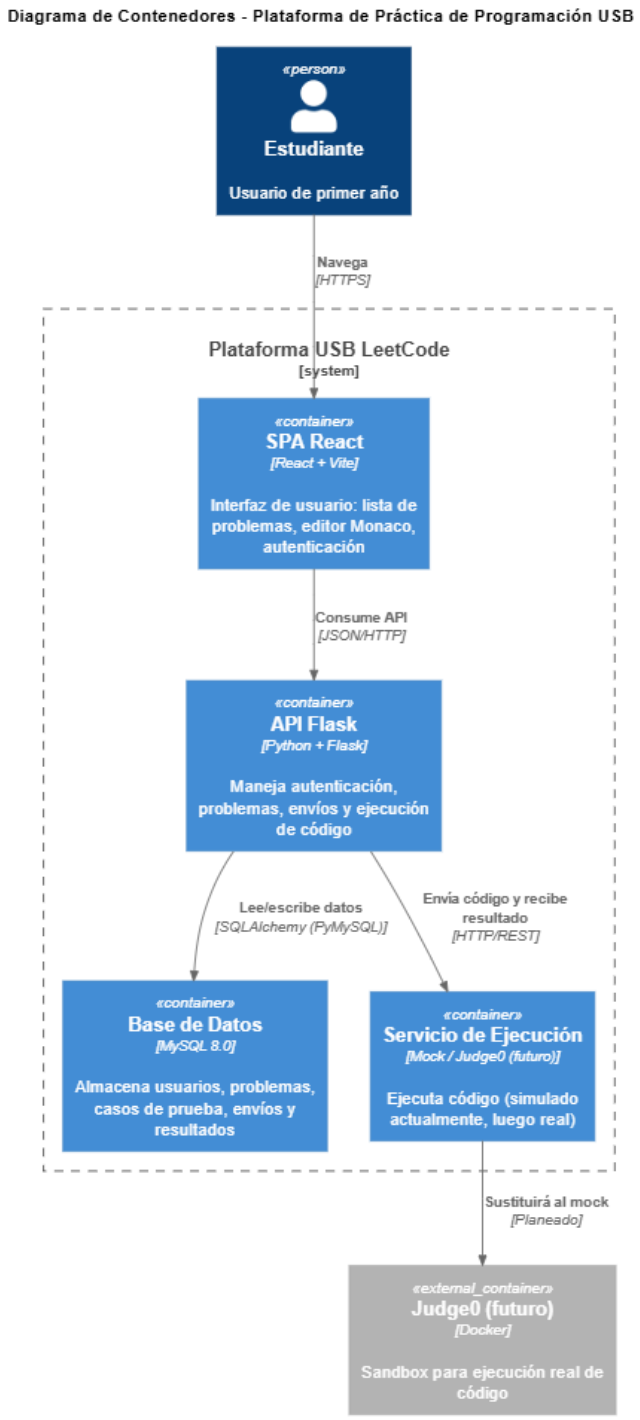
Autores
David Steven Hernandez Garces – Estudiante de Ingeniería de Sistemas, USB Cali

6. Diagramas

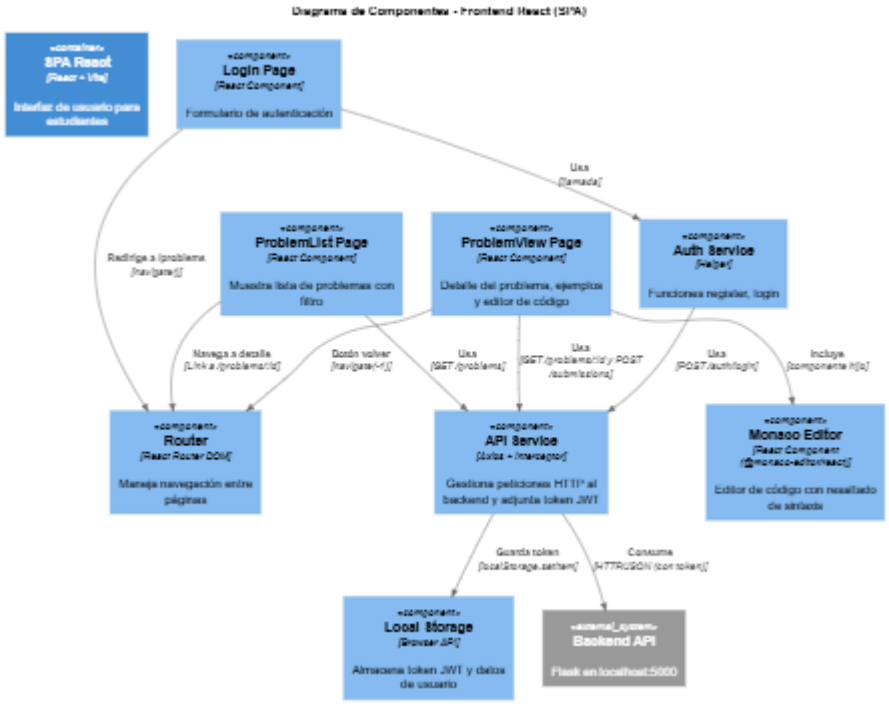
Contexto



Contenedores



Componentes frontend



Componentes Backend

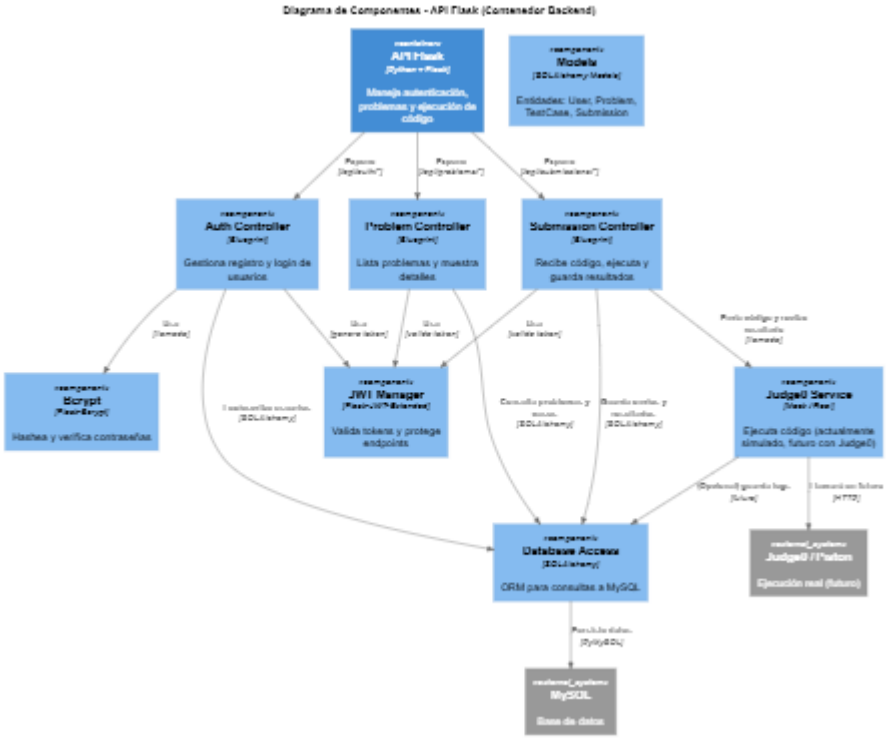
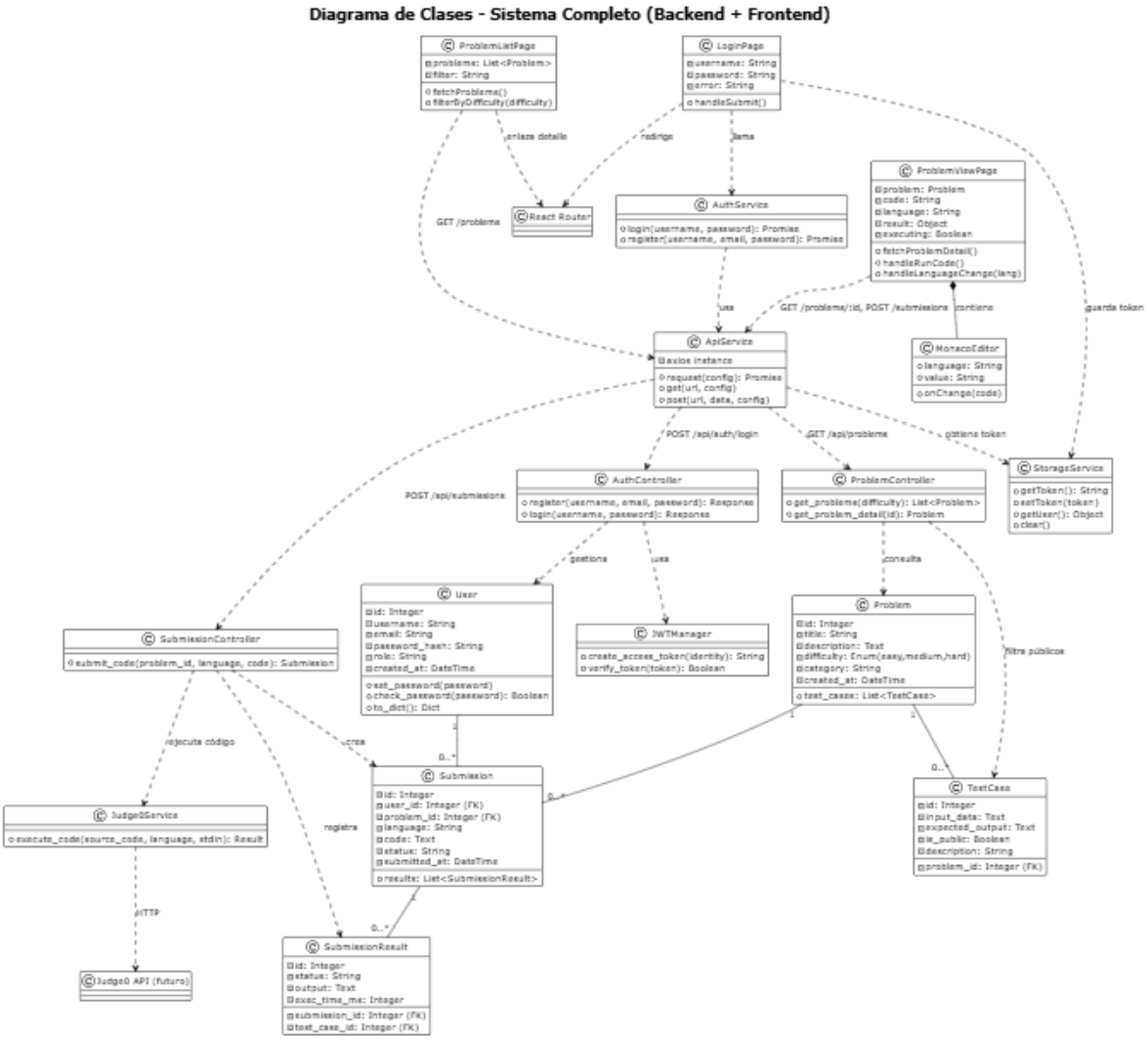


Diagrama código



7. Tablero Kanban

https://app.clickup.com/9013086219/v/s/901313749846

2 collapsed

IN DESIGN 2

- Crear modelos SQLAlchemy — Normal — backend, database
- HU4: Mostrar casos públicos en detalle del problema — Normal — frontend, testing

+ Add Task

IN DEVELOPMENT 5

- Configurar proyecto frontend (React + Vite) — High — frontend, setup
- Inicializar entorno virtual, instalar dependencias (Flask,... — High — backend, setup
- HU1: Backend de registro y login con JWT — High — autenticación, backend
- HU2: Backend de listado y detalle de problemas — High — backend, problemas
- HU3: Frontend – editor Monaco y botón ejecutar — Normal — ejecución, frontend

+ Add Task

IN REVIEW 2

- HU1: Frontend de login y protección de rutas — High — autenticación, frontend
- HU3: Backend – endpoint de envío simulado — Normal — backend, ejecución

+ Add Task

TESTING 0

READY FOR DEVELOPMENT 7

- Configurar repositorio GitHub — High — devops
- HU2: Frontend – lista de problemas y filtro — High — frontend, problemas
- Redactar ADRs (4 decisiones) — documentación
- Diagramas C4 (código PlantUML) — arquitect..., documentac...
- Definition of Done — documentación, proceso
- README.md del proyecto — documentación
- Poblar base de datos con seed — database, testing

SHIPPED 0

CANCELLED 1

- Integración real con Judge0 — diferido, ejecución

+ Add Task